

Задание 5

Методы Монте-Карло

Отчет

Загоруйко Т.Е.

tzag2001@gmail.com

2024

1. Постановка задачи

Требуется написать параллельную программу, реализующую метод Монте-Карло для модели случайных блужданий.

Программа принимает на вход через аргументы командной строки шесть параметров:

- a, b — границы интервала,
- p — вероятность перехода частицы вправо,
- x — начальная позиция,
- N — число частиц,
- P — число потоков.

2. Формат командной строки

`OMP_NUM_THREADS=16 ./task <граница a> <граница b> <вероятность перехода частицы вправо> <начальная позиция> <число частиц> <число потоков>`

3. Спецификация системы

Вычисления производились на суперкомпьютере Polus. По результатам команды `lscri`:

```
[edu-cmc-sqi22-26@polus-ib sem05]$ lscpu
Architecture:      ppc64le
Byte Order:        Little Endian
CPU(s):            152
On-line CPU(s) list: 0-151
Thread(s) per core: 8
Core(s) per socket: 9
Socket(s):         2
NUMA node(s):      2
Model:             1.0 (pvr 004c 0100)
Model name:        POWER8NVL (raw), altivec supported
CPU max MHz:       4023.0000
CPU min MHz:       2061.0000
L1d cache:         64K
L1i cache:         32K
L2 cache:          512K
L3 cache:          8192K
NUMA node0 CPU(s): 0-71
NUMA node1 CPU(s): 72-151
```

4. Результаты выполнения

Результатом программы являются три числа: вероятность достижения b , среднее время жизни одной частицы, время работы основного цикла.

Компиляция производилась следующей командой:

```
xlcc++_r -qsmp=omp -Wall -Wextra -g -O3 task.cpp -o task
```

Далее был создан файл `suffering.lsf` со следующими командами:

```
source /polusfs/setenv/setup.SMPI
#BSUB -m "polus-c4-ib"
#BSUB -n 1
#BSUB -W 00:10
#BSUB -o task.%J.out
#BSUB -e task.%J.err
OMP_NUM_THREADS=16 mpiexec ./task -10 1000 0.75 105 100000
```

<num_threads>

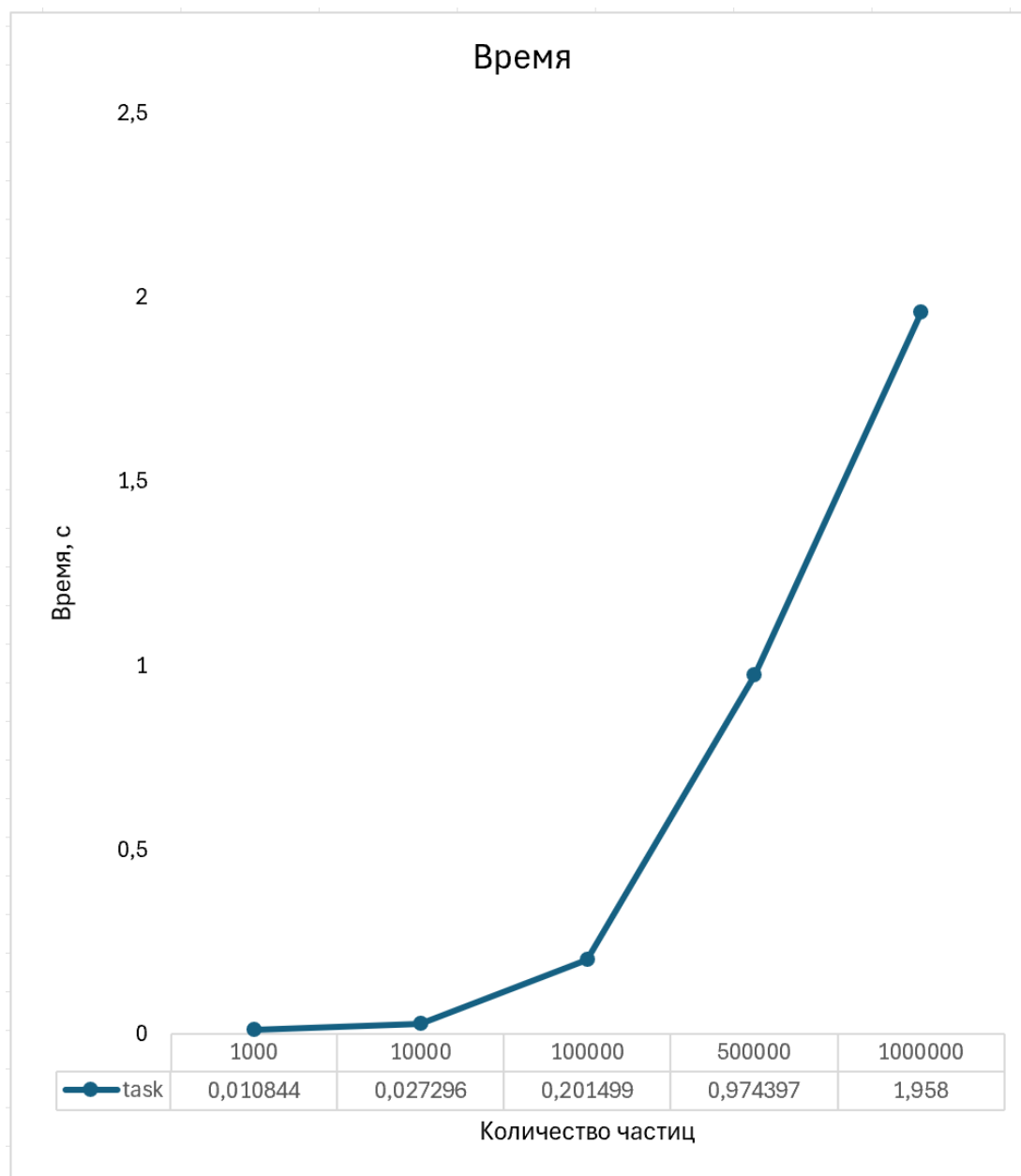
Далее отправка в очередь производилась командой:

```
bsub < suffering.lsf
```

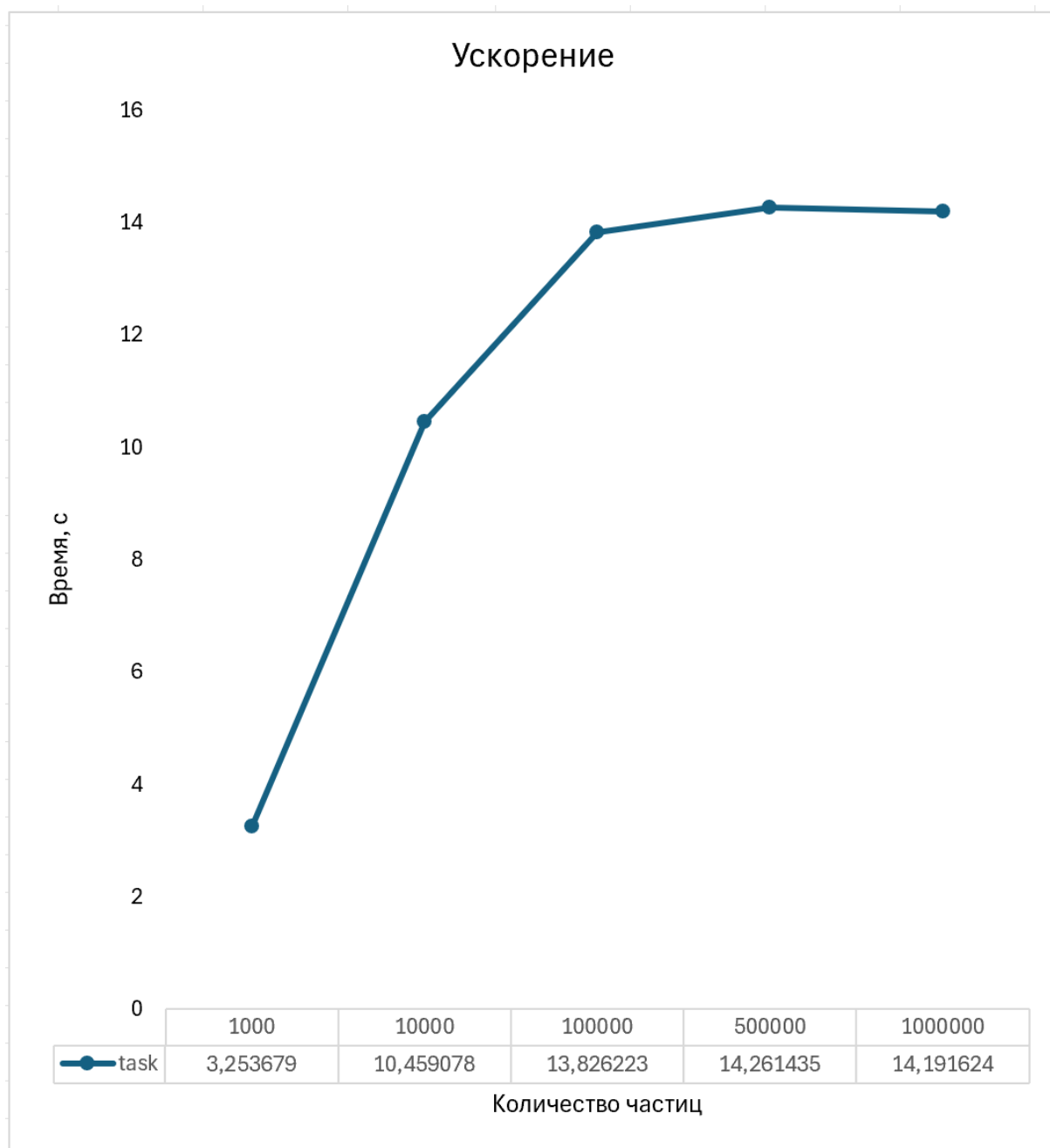
4.1. Графики при фиксированном значении P (процессы)

Для данного подраздела P фиксировано и равно 16.

На графике ниже представлена зависимость времени выполнения программы от числа частиц.

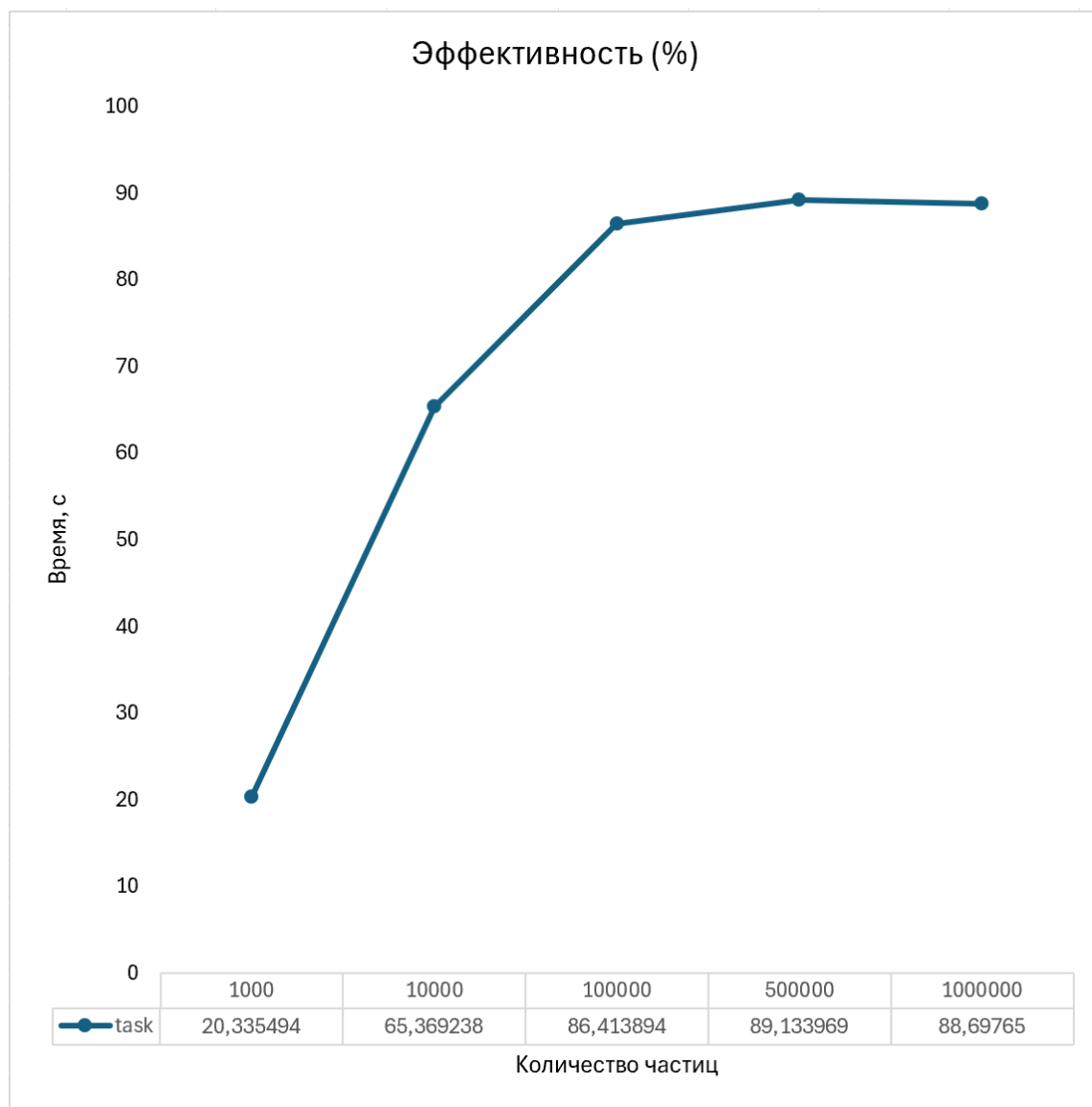


На графике ниже представлено ускорение программы при разном числе частиц:



На графике ниже представлена эффективность в процентах. Высчитывалась по формуле:

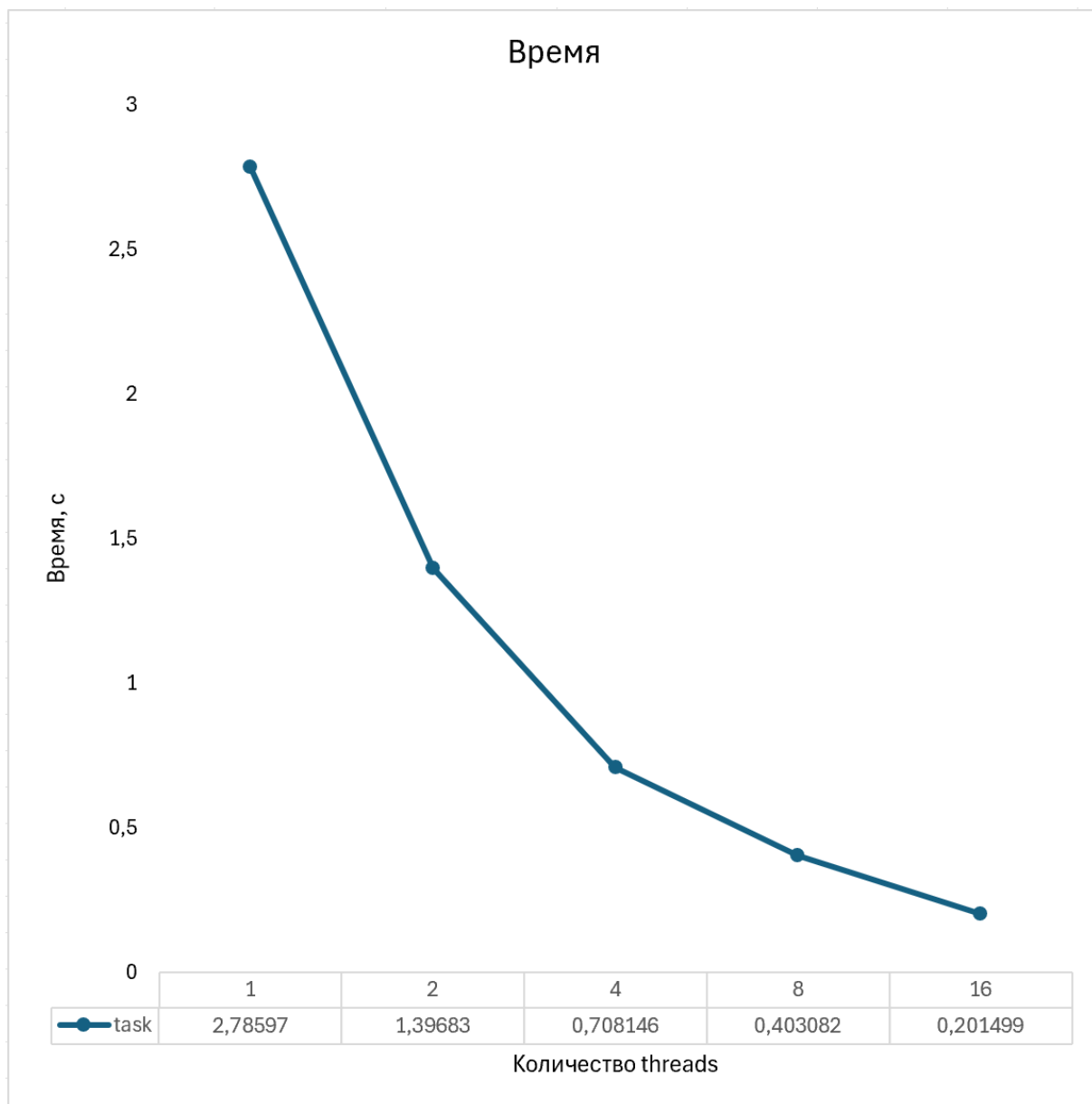
$$\text{Эффективность} = \frac{\text{Ускорение}}{P} \times 100$$

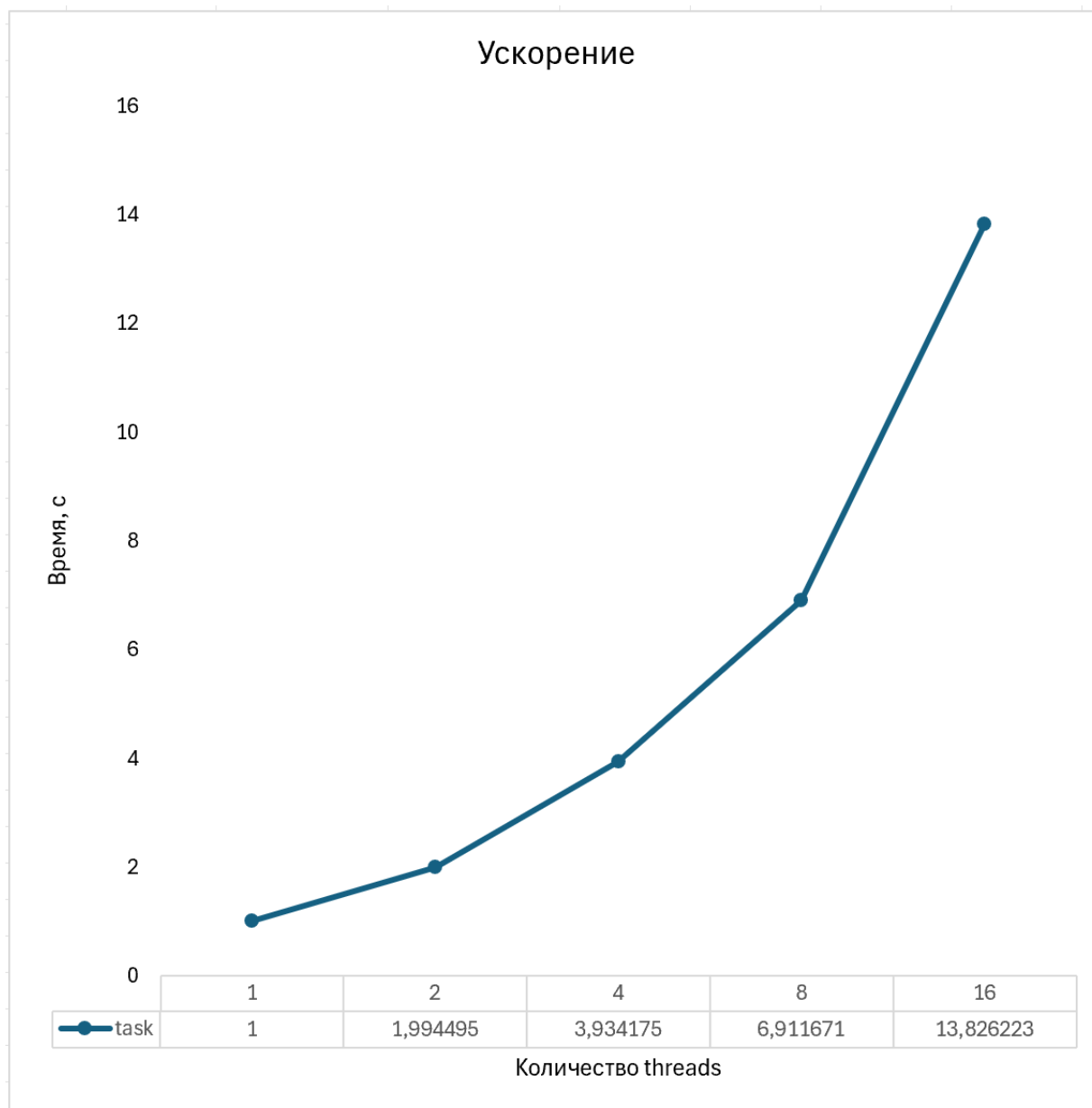


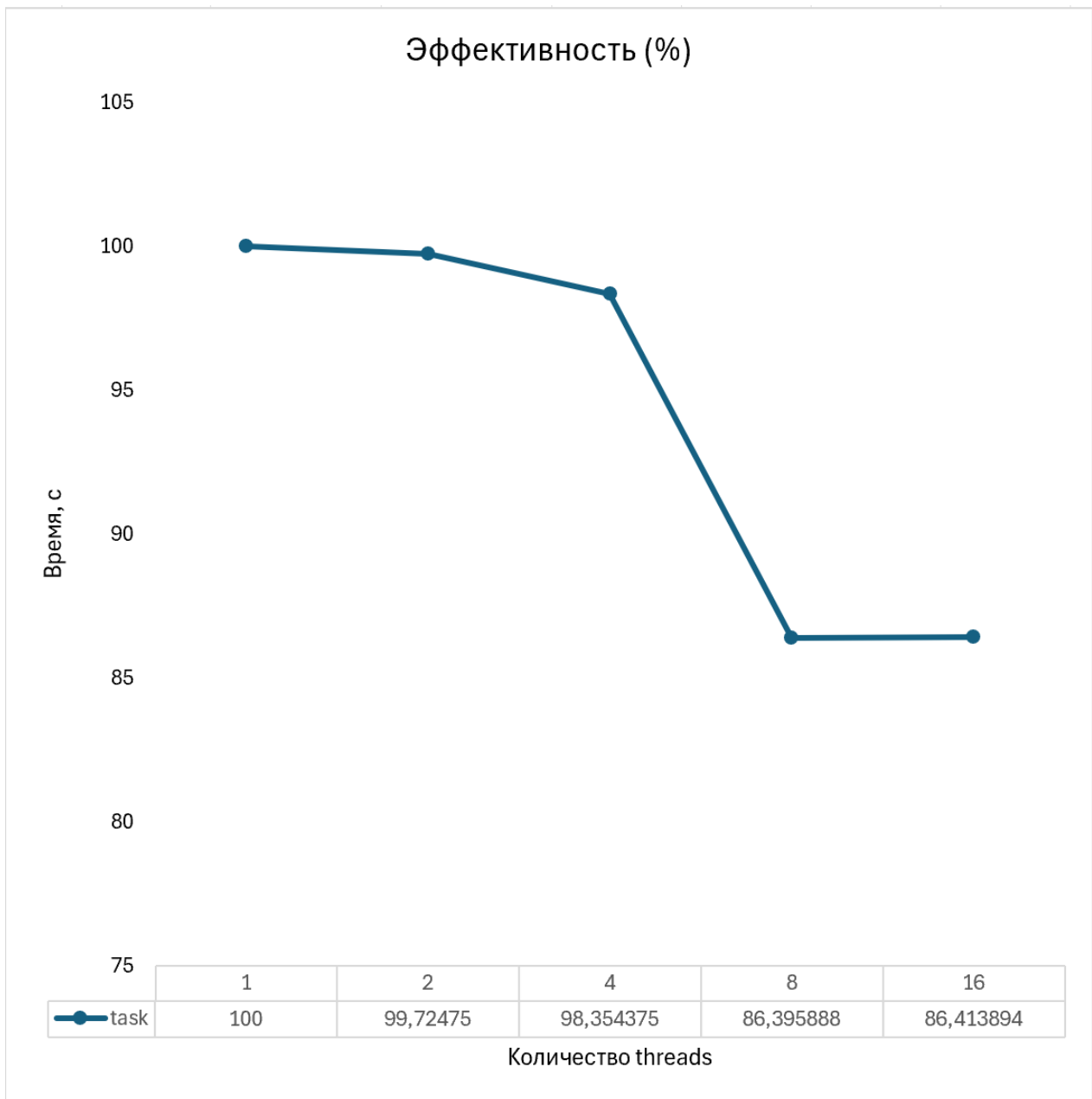
4.2. Графики при фиксированном значении N

Для данного подраздела N фиксировано и равно 100000.

На графиках ниже представлены зависимость времени выполнения программы, ускорение и эффективность в зависимости от запусков на разном числе процессов.







Эффективность вычислялась по аналогичной формуле, как и в подразделе 4.1.